

How FluencyOS is tested

The whole testing structure, and what every test file does — written for someone new to testing.

Layer	Tool	Tests	Where
Type-check	TypeScript (tsc)	—	renderer/src (all code)
Frontend unit tests	Vitest	49	renderer/src/**/*.test.ts
Backend tests	pytest	989	backend/tests/
UI tests	Selenium + pytest	9	ui-tests/
Total		1047	

Contents

- 1. What a test is
- 2. The four layers, and why there are four
- 3. The folders and files at a glance
- 4. How the test machinery works (fixtures, fake services, helpers)
- 5. Running the tests, the report, and GitHub Actions
- 6. The beginner tests
- 7. Every backend test file, by feature
- 8. Every frontend test file
- 9. The UI (Selenium) test files
- 10. Glossary

1. What a test is

A **test** is a small piece of code that runs part of the app and checks that the result is what we expect. If someone later changes the app and breaks that part, the test fails and points at the exact line. Tests catch mistakes automatically, before a user does.

Every test in this project follows the same three steps:

```
def test_550_words_count_as_2_pages():
    words_read = 550
    pages = pagination.pages_from_words(words_read)
    assert pages == 2
```

Arrange: set up the input
Act: run the code
Assert: check the result

assert means “this must be true”. If pages were 3, the test fails and pytest prints `assert 3 == 2` — what the app returned, next to what was expected.

In TypeScript (Vitest) the same idea is written with `expect(...).toBe(...)`:

```
it('shows 65 seconds as 1:05', () => {
  expect(timecode(65_000)).toBe('1:05');
});
```

2. The four layers

FluencyOS has two halves: a **backend** in Python (FastAPI with a SQLite database — it stores words, books and progress, and runs the AI) and a **frontend** in TypeScript/React (the screens), shown inside Electron as a desktop app. Each half is checked on its own, then the two are checked together.

#	Layer	What it checks	Speed
1	Type-check	The TypeScript code has no type mistakes (e.g. a number passed where text is expected). Nothing is run.	~10 s
2	Frontend unit tests	Small frontend functions on their own, e.g. “65 s shows as 1:05”.	~2 s
3	Backend tests	Python functions on their own (unit tests), and the real API over HTTP with a real database (integration tests).	~2–3 min
4	UI tests	The whole app in a real Chrome window, clicked through like a person would.	~25 s

The testing pyramid

Most tests are at the bottom: fast unit tests, each checking one function. They run in milliseconds and point straight at the broken function. Fewer tests sit higher up: they are slower, but prove the parts work together.

	UI tests (9)	few · slow · test everything together
	API / integration tests	backend + database together
	Unit tests	many · very fast · one function each

3. The folders and files at a glance

Path	What it is
backend/tests/	The backend tests: 60 files, 978 tests, grouped by feature (section 7).
backend/tests/conftest.py	Shared setup for every backend test. Gives each test a fresh database, an HTTP client and the secret token (section 4).
backend/tests/levels.py	A small helper that gives a test learner a CEFR level directly, without taking the level test.
backend/tests/beginner/	The 11 simple, heavily commented backend tests (section 6).
backend/pyproject.toml	pytest settings, e.g. only keep temporary files from tests that failed.
renderer/src/**/*.test.ts	The frontend tests: 7 files, 49 tests, each next to the code it tests (section 8).
renderer/vitest.config.ts	Vitest settings: where to find the frontend tests.
ui-tests/conftest.py	Starts the real backend and interface, and opens Chrome for each test (section 4).
ui-tests/helpers.py	type_into, click, click_menu and sees — make UI tests read like instructions.
ui-tests/test_*.py	The 9 Selenium UI tests (section 9).
scripts/test-all.mjs	npm test: runs all four layers in order, prints a summary and writes the HTML report.
scripts/test-beginner.mjs	npm run test:beginner: runs only the beginner tests, printing each by name.
.github/workflows/tests.yml	GitHub Actions: runs npm test on GitHub's servers after every push and keeps the report.
test-reports/	Output: index.html (the report), JUnit XML files and coverage data. Created by npm test.
docs/beginner-testing.md	A short guide to the beginner tests.

4. How the test machinery works

Fixtures: setup that pytest does for you

A **fixture** is setup code that pytest runs before a test. A test asks for one simply by naming it as a parameter. They live in `conftest.py` files, which pytest finds automatically.

```
def test_a_saved_word_appears_in_the_vocabulary_list(client, auth_headers):  
    ... # pytest has already made `client` and `auth_headers` for this test
```

Fixture	File	What it gives the test
client	backend/tests/conftest.py	A way to send HTTP requests to the backend, connected to a brand-new database for this test only.
auth_headers	backend/tests/conftest.py	The secret token the app sends with every request. Without it the backend answers 401 Unauthorized.
_started_db	backend/tests/conftest.py	A fully set-up database, made once and copied for each test — much faster than building it every time.
tmp_path	built into pytest	An empty temporary folder, deleted afterwards.
monkeypatch	built into pytest	Temporarily replaces a function with a fake one (see below).

Fixture	File	What it gives the test
app_servers	ui-tests/conftest.py	Starts the real backend (empty database) and the real interface once for all UI tests, and stops them after.
browser	ui-tests/conftest.py	A fresh Chrome window with FluencyOS open.
api	ui-tests/conftest.py	Calls the backend directly, to set up data quickly (e.g. save a word).
learner	ui-tests/conftest.py	A learner who has finished onboarding, already signed in in the browser.

Why every test gets its own database

Tests must be **independent**: a word saved in one test must never show up in another, and tests must pass in any order. They also must never touch your real data. So each test works on its own copy of a database in a temporary folder, which is thrown away afterwards.

Fake services (mocking)

Some parts of the app talk to the internet (the online dictionary, cloud AI) or to large AI models that take gigabytes of memory. Tests must not depend on those: they would be slow, cost money, and fail whenever the Wi-Fi is down. So in tests those calls are replaced with fakes that return a fixed answer, using pytest's monkeypatch. This is called **mocking**. The test then checks what our code does with that answer.

```
monkeypatch.setattr(dictionary_lookup, "search", lambda word: [fake_result])
# now the app "looks up" words without going online
```

Running the whole app for UI tests

FluencyOS normally runs inside Electron, which gives the screens a small “bridge” to the desktop (where the backend is, the computer's hardware, file pickers). Chrome has no such bridge, so `ui-tests/conftest.py` puts a simple stand-in in place before the page loads. Then Selenium types and clicks. Every helper in `helpers.py` waits up to 10 seconds for the screen to catch up, because data arrives from the backend a moment later — forgetting to wait is the most common cause of “flaky” UI tests that sometimes pass and sometimes fail.

```
type_into(browser, "Enter your name", "Ana")
click(browser, "Continue")
assert sees(browser, "Where you are now")
```

5. Running the tests

Command	What it runs	Time
<code>npm run test:beginner</code>	The 24 beginner tests from all layers, each printed by name. Chrome is visible.	~30 s
<code>npm run test:ui</code>	Only the Selenium tests, in a visible Chrome window.	~25 s
<code>npm test</code>	Everything: all four layers. Chrome runs hidden (headless).	~3–4 min
<code>HEADLESS=1 npm run test:ui</code>	The UI tests without opening a window.	~25 s

What npm test prints

```
✓ TypeScript          no type errors
✓ Frontend (Vitest)    49/49 passed
✓ Backend (pytest)     989/989 passed · 88.1% line coverage
✓ UI (Selenium)        9/9 passed

1047 of 1047 tests passed
```

The backend tests run **in parallel**, spread over every CPU core (pytest-xdist, `-n auto`), which is why about a thousand tests finish in a couple of minutes.

The report

`npm test` writes `test-reports/index.html`. Open it in a browser to see every test grouped by feature, how long each took, and any failures with their messages.

Code coverage

Line coverage (about 88% for the backend) is the share of the backend’s lines of code that ran during at least one test. It shows which code is tested at all — not whether it is tested well — but a low number points at areas nobody has tested.

GitHub Actions (continuous integration)

`.github/workflows/tests.yml` tells GitHub to run the same `npm test` on its own servers after every push and pull request: it installs `ffmpeg` (for the video tests), Node and Python dependencies, runs the tests, and keeps the HTML report as a download. So even if someone forgets to run the tests, a failure shows up on GitHub. This practice is called **continuous integration (CI)**.

When a test fails

pytest shows the failing line and both values:

```
> assert pages == 3
E      assert 2 == 3
```

Read it as: “the app returned 2, but the test expected 3”. Then decide which is wrong — the app (fix the code) or the test (the expected behaviour changed on purpose, so update the test).

6. The beginner tests

A small set written for learning and demonstrating, with Arrange / Act / Assert comments. Run them with `npm run test:beginner`.

File	Kind	Tests
backend/tests/beginner/test_1_simple_functions.py	Unit tests: page counting, the level-test pass mark, the “no jumping up a level” rule, splitting AI replies into sentences	5
backend/tests/beginner/test_2_api_basics.py	API tests: health check, token security, creating a learner, saving a word, a word becoming a flashcard, the level rule via HTTP	6
renderer/src/tests-beginner/simple.tests	Frontend unit tests: video time display, clicking subtitle words, quiet hours	4
ui-tests/	UI tests in Chrome (section 9)	9

7. Every backend test file, by feature

All in backend/tests/. The number is how many tests pytest runs from the file (a test repeated for several inputs counts once per input). “Faked” means the online service or AI model is replaced with a fake in that file (section 4).

App basics, security and settings · 7 files · 66 tests

File	What it checks	Tests
test_health.py	The backend answers “are you running?” — and refuses anyone without the app’s secret token.	2
test_onboarding_routes.py	Onboarding end to end over the API: create a learner, save their choices, and check an API key is never stored in plain text.	4
test_profile_settings.py	Editing your profile after onboarding: name, languages, level and profile picture.	13
test_settings_routes.py	The Settings page’s endpoints: reading and changing only the settings that were sent.	12
test_hardware_capability.py	Onboarding’s AI model recommendation: always a model the app really has, one that fits in memory, and not one that would reply slowly.	16
test_hardware_routes.py	The same recommendation, called over HTTP.	2
test_compute.py	Whether the AI models run on the CPU or the GPU, and that each engine obeys the setting.	17

English level (CEFR A1-C2) · 5 files · 79 tests

File	What it checks	Tests
test_level_test.py	The level test: questions are drawn fairly, marked on the server, and passing is the only way up a level.	22
test_level_routes.py	The reading-level and session endpoints.	15
test_leveling_rules.py	The rules that decide when a learner’s level should change.	14
test_cefr_lexicon.py	Looking up how hard an English word is (its CEFR level) from the word list.	21
test_vocabulary_cefr.py	Saved words get the right CEFR level attached.	7

Learn by reading (books) · 20 files · 206 tests

File	What it checks	Tests
test_txt_parser.py	Reading plain .txt books: paragraphs, empty files, unusual text encodings.	4
test_epub_parser.py	Reading EPUB books: title, author, chapters and cover.	6
test_mobi_parser.py	Reading Kindle MOBI/AZW3 books.	8
test_pdf_parser.py	Reading PDF books: text, pages and headings.	19
test_ingest_pipeline.py	Importing a book from start to finish — and that a broken file leaves nothing half-saved behind.	5
test_books_routes.py	The book endpoints: import, list, re-import the same file, missing files.	19
test_book_finished.py	Marking a book as finished.	7
test_book_search.py	Building safe search queries from what the learner types.	10
test_search_routes.py	Searching inside a book returns the right paragraph with a preview snippet.	7
test_highlights_routes.py	Highlighting text in a book, and rejecting highlights that point nowhere.	10
test_pagination.py	Splitting a book into pages (275 words per page).	10
test_native_pages_routes.py	PDFs keep their own printed page numbers.	8
test_page_text_layer.py	The invisible, selectable text laid over a PDF page image, so words can be clicked.	30
test_page_heat.py	Colouring hard words on a PDF page, in the page's own coordinates.	9
test_difficulty_heat.py	Deciding which words are hard for this learner (the “heat” colours).	10
test_jump_targets.py	Table-of-contents links land on the page they say they do.	5
test_reader_prefs.py	Reader display preferences such as font size and theme.	9
test_reading_goal.py	The daily page goal, the reading streak and the bookshelf.	10
test_reading_routes.py	The reading endpoints: difficulty colours and offline word look-up.	15
test_vocab_context_page.py	A saved word remembers which page it came from.	5

Learn by watching (videos) · 5 files · 121 tests

File	What it checks	Tests
test_media_routes.py	Learn by watching end to end, with a real video file: import, subtitles, clips.	44
test_media_subtitles.py	Reading subtitle files and the maths of cutting a clip around a line.	23
test_media_generate.py	Creating subtitles automatically when a video has none.	6
test_media_optimize.py	Preparing an MP4 so it can be skipped around instantly.	11
test_stt_windowing.py	Transcribing long videos in overlapping pieces without losing or doubling words.	37

Vocabulary and the dictionary · 7 files · 126 tests

File	What it checks	Tests
test_vocabulary.py	Saving words, and the flashcard that is created with each one.	26
test_vocabulary_manual_add.py	Adding a word by hand from a dictionary search (the online dictionary is faked).	18
test_vocabulary_browse.py	The Vocabulary page: search, filter, sort and the overview numbers.	39
test_vocabulary_ai.py	AI help for a word — explanations, examples, memory tricks (the AI is faked).	11
test_dictionary_casing.py	Capital letters in look-ups: “London” and “london” are handled correctly.	8
test_dictionary_synonyms.py	Filtering out poor-quality synonyms.	5
test_pronunciation.py	Pronunciation (IPA) for words, offline.	19

Review and progress · 4 files · 82 tests

File	What it checks	Tests
test_fsrs.py	The spaced-repetition scheduler (FSRS) that decides when each flashcard is due next.	15
test_review.py	The review queue: which cards come up, rating an answer, undo and suspend.	43
test_activity.py	The dashboard’s activity calendar agrees with what was really read and reviewed.	7
test_forest.py	The Forest: the picture of how well each word is known.	17

Conversation practice and AI · 10 files · 241 tests

File	What it checks	Tests
test_conversation.py	Conversation sessions: turns, speech in, speech out, reports (the AI and voice models are faked).	66
test_conversation_report.py	The report after a conversation: its measurements and the AI’s feedback format.	23
test_conversation_scenarios.py	The conversation scenes, the instructions each one gives the AI, and learners’ own scenes.	31
test_chat_history.py	Formatting the conversation so every AI provider accepts it.	15
test_target_words.py	Which saved words a conversation asks the learner to use.	8
test_cloud_llm.py	The OpenRouter cloud AI option (network calls faked).	21
test_gemini_llm.py	The Google Gemini cloud AI option (network calls faked).	13
test_json_salvage.py	Recovering usable answers when a small AI model returns slightly broken JSON.	12
test_tts_engines.py	Choosing between the two text-to-speech voice engines.	27
test_tts_voices.py	Choosing the voice the AI speaks in.	25

Scene Description Challenge · 2 files · 57 tests

File	What it checks	Tests
test_challenge.py	The picture-description game: scenes, hints, difficulty and marking.	49
test_challenge_routes.py	The same challenge, over HTTP.	8

8. Every frontend test file

Vitest tests live next to the code they test, named `*.test.ts`. They test the parts of the interface written as plain functions, so no browser is needed.

File (in renderer/)	What it checks	Tests
src/features/conversation/spokenTiming.test.ts	Highlighting the word the AI voice is currently saying.	6
src/features/conversation/vadGate.test.ts	Hands-free mode: deciding when the learner has started and stopped talking.	11
src/features/player/subtitleTokens.test.ts	Splitting subtitle lines into clickable words, and video timecodes.	8
src/features/reader/readerHelpers.test.ts	PDF reader: finding the sentence around a word, search highlights, the contents panel.	10
src/lib/friendlyError.test.ts	Turning technical errors into messages a learner can act on.	6
src/lib/quietHours.test.ts	When the review reminder must stay silent (including windows past midnight).	4
src/tests-beginner/simple.test.ts	Beginner examples: video time display, clicking subtitle words, quiet hours.	4

9. The UI (Selenium) test files

File	What it does
ui-tests/conftest.py	Starts the backend and interface on free ports with an empty database, opens Chrome, adds the Electron stand-in; provides the browser, api and learner fixtures. Stops everything afterwards.
ui-tests/helpers.py	<code>type_into</code> (type into a box), <code>click</code> (a button), <code>click_menu</code> (the left menu), <code>sees</code> (text appears on screen). Each waits up to 10 s.
ui-tests/test_1_onboarding.py (6)	A new learner: the first screen shows, a name is required, “I’m a beginner” sets A1, the AI step recommends a model, and finishing opens the main app.
ui-tests/test_2_main_screens.py (3)	A signed-in learner: Review says “Nothing saved yet”, a saved word appears on the Vocabulary page, and it is then ready to review.

The UI tests choose the cloud AI option during onboarding, so they never start a multi-gigabyte model download.

10. Glossary

Term	Meaning
Test	Code that runs part of the app and checks the result.
Assertion	The check itself: assert (Python) or expect (TypeScript). If it is false, the test fails.
Unit test	Tests one function on its own. Fast.
Integration / API test	Tests several parts together, e.g. the API plus a real database.
UI / end-to-end test	Uses the whole app through its screens, like a person.
Fixture	Setup pytest runs for a test, requested by naming it as a parameter.
Mock / fake	A stand-in for something slow or external (internet, AI), returning a fixed answer.
pytest	The Python testing tool used for the backend and UI tests.
Vitest	The TypeScript testing tool used for frontend tests.
Selenium	Controls a real web browser from code: typing, clicking, reading the screen.
Headless	A browser running without a visible window.
Flaky test	A test that sometimes passes and sometimes fails without any code change — usually a timing problem.
Coverage	The share of code lines that ran during the tests.
CI (continuous integration)	Running the tests automatically on a server after every change (here: GitHub Actions).
Status code	The number an API answers with: 200 OK, 201 Created, 401 Unauthorized, 403 Forbidden, 404 Not Found.